PLACEMENT INTERVIEW REVISION

Matplotlib Complete Revision Guide

Every topic, simple explanation, diagrams, code and real interview questions — in one place.

21 Topics | Diagrams | Code Snippets | Interview Traps | 30 Q&A

HOW TO USE THIS DOC

Before solving questions: read the topic + its "In interview, say this" box. That is enough to answer 90% theory questions.

Night before interview: read only the green boxes, the tables, and Topic 20 (cheat sheet) + Topic 21 (Q&A).

Revision notes — Data Visualization with Matplotlib · Companion doc: **Seaborn Complete Revision Guide**

Contents

1 What is Matplotlib + Architecture
2 Anatomy of a Plot (Figure, Axes, Axis)
3 Two Interfaces: pyplot vs Object-Oriented
4 Line Plot + All Customization
5 Scatter Plot
6 Bar Chart (grouped, stacked, horizontal)
7 Histogram
8 Pie Chart
9 Box Plot & Other Statistical Plots
10 Which Chart to Use When
11 Subplots & Layout (4 ways + GridSpec)
12 Colors, Colormaps, Styles, rcParams
13 Text, Annotation & Legend Control
14 Axis Control: limits, ticks, spines, twin, log
15 Images & Heatmap with imshow
16 3D Plotting
17 Saving Figures, DPI & Backends
18 Animation (FuncAnimation)
19 Matplotlib vs Seaborn vs Plotly
20 Common Errors + Cheat Sheet
21 Real Interview Questions & Answers

▸ The 6 things interviewers always ask about Matplotlib

#	Question area	Where in this doc
1	Difference between Figure, Axes and Axis	Topic 2
2	pyplot interface vs Object-Oriented interface	Topic 3
3	Which chart for which type of data	Topic 10
4	How to make subplots / multiple charts in one figure	Topic 11
5	Histogram vs Bar chart (most common trap)	Topic 7
6	Matplotlib vs Seaborn — when to use which	Topic 19

1 What is Matplotlib + Architecture

Matplotlib is the base plotting library of Python. Almost every other library (Seaborn, Pandas `.plot()`, Scikit-learn plots) is built on top of it.

▸ What is it?

- A 2D (and basic 3D) plotting library for Python, made by **John Hunter in 2003**.
- It was made to copy MATLAB's plotting style in Python — that is why the module is called `pyplot`.
- It gives **full low-level control**: every line, tick, colour and label can be changed.

▸ Why do we need data visualization at all?

- Numbers alone hide patterns. A chart shows trend, outlier and relation in one look.
- Classic proof: **Anscombe's Quartet** — 4 datasets with same mean, same variance, same correlation, but totally different charts.
- Used in EDA (exploratory data analysis), model result checking, and reporting to business people.

▸ Architecture: 3 layers

This is a favourite theory question. Matplotlib is built in 3 layers.

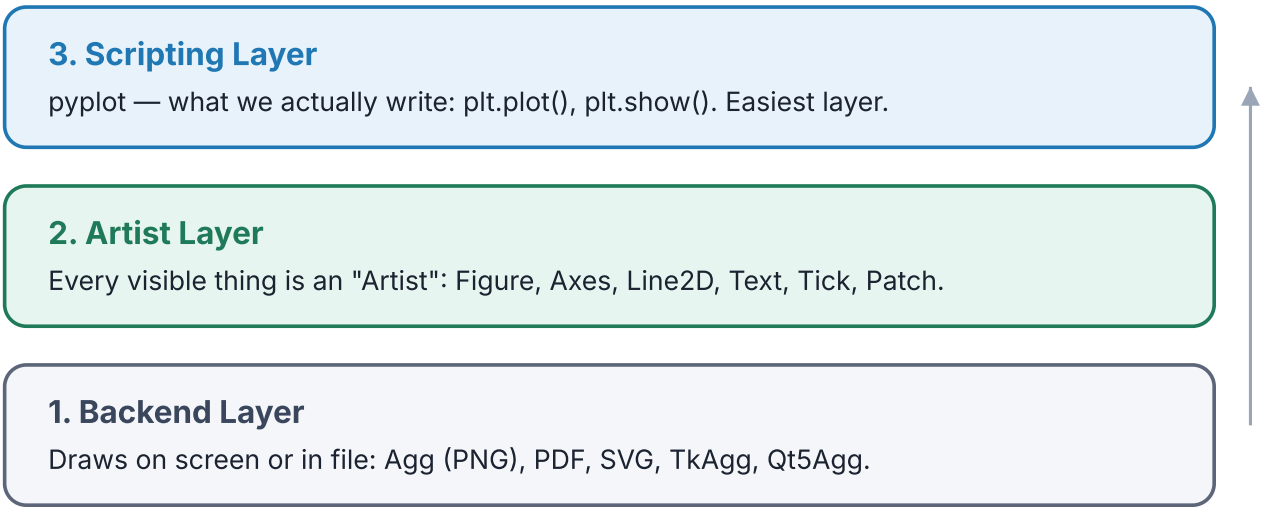


Figure 1.1 — We write in the top layer; the bottom layer does the real drawing.

IN INTERVIEW, SAY THIS

"Matplotlib has three layers. **Backend** layer actually renders the output (Agg for PNG, PDF for files). **Artist** layer is the object layer — every element like Figure, Axes, Line2D, Text is an Artist object. **Scripting** layer is `pyplot`, a simple wrapper we use for quick plotting."

▸ Import convention

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
%matplotlib inline      # in Jupyter: show chart inside notebook
```

2 Anatomy of a Plot (Figure, Axes, Axis)

If you remember only one diagram from this doc, remember this one. Interviewers ask "Figure vs Axes vs Axis" very often.

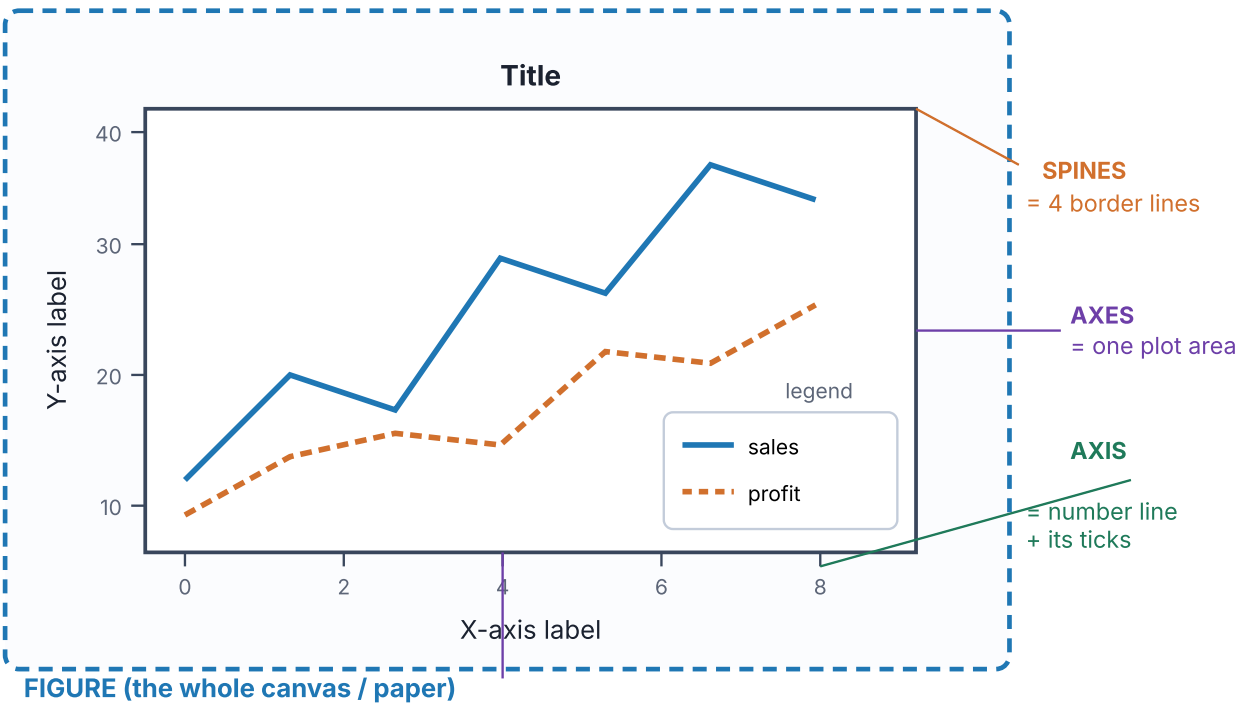


Figure 2.1 — One Figure can hold many Axes. One Axes has 2 Axis objects (x and y).

Term	Simple meaning	Code
Figure	The full page / canvas. Container of everything. You save a Figure, not an Axes.	`fig = plt.figure()`
Axes	One actual plot (the box where data is drawn). A figure can have many Axes. **Axes = subplot.**	`ax = fig.add_subplot()`
Axis	The number line (x-axis / y-axis). Controls limits, ticks, scale.	`ax.xaxis`
Artist	Anything you can see on the figure — line, text, tick, patch, legend.	`Line2D`, `Text`
Spines	The 4 border lines of the Axes box (top/bottom/left/right).	`ax.spines['top']`

INTERVIEW TRAP

Axes is NOT plural of Axis. "Axes" means one complete plot area. "Axis" means one number line. Figure > Axes > Axis is the hierarchy.

ONE-LINE ANSWER TO REMEMBER

"Figure is the paper, Axes is the graph drawn on that paper, and Axis is the ruler on the side of that graph."

3 Two Interfaces: pyplot vs Object-Oriented

Matplotlib can be used in two styles. Interviewers love this question because most freshers know only the first one.

▸ Style 1 — pyplot / state-machine style

You never store the figure. `plt` keeps a hidden "current figure" and "current axes" in memory and every command goes there.

```
plt.figure(figsize=(8,4))
plt.plot(x, y)
plt.title('Sales')
plt.xlabel('Month')
plt.show()
```

▸ Style 2 — Object-Oriented (OO) style

You get objects in variables and call methods on them. This is the **recommended professional way**.

```
fig, ax = plt.subplots(figsize=(8,4))   # fig = Figure, ax = Axes
ax.plot(x, y)
ax.set_title('Sales')
ax.set_xlabel('Month')
plt.show()
```

Point	pyplot style	Object-Oriented style
Who holds state	Hidden — "current axes"	You hold it in `ax` variable
Writing speed	Very fast, less typing	Slightly more typing
Many subplots	Confusing, you must keep switching	Clean — `ax[0]`, `ax[1]`
Inside functions / libraries	Risky, can plot on the wrong figure	Safe, you pass `ax` as argument
Method names	`plt.title()`, `plt.xlabel()`, `plt.xlim()`	`ax.set_title()`, `ax.set_xlabel()`, `ax.set_xlim()`
Use for	Quick EDA in notebook	Dashboards, reports, production code

IN INTERVIEW, SAY THIS

" `pyplot` is a state-machine interface — it keeps track of the current figure and axes, so commands apply implicitly. The object-oriented interface gives me the Figure and Axes objects directly, so I can control each subplot explicitly. For quick EDA I use pyplot, but for multiple subplots or reusable plotting functions I always use the OO interface."

REMEMBER THE NAMING RULE

Almost every `plt.xyz()` becomes `ax.set_xyz()` in OO style. Exceptions: `plt.plot → ax.plot`, `plt.legend → ax.legend`, `plt.scatter → ax.scatter` (plotting functions keep the same name).

▸ Figure creation: 3 common ways

```
fig, ax = plt.subplots()                # 1 figure + 1 axes  (BEST)
fig, ax = plt.subplots(2, 2)            # 2x2 grid, ax is a 2D numpy array

fig = plt.figure()                      # empty figure
ax  = fig.add_subplot(1, 1, 1)          # add axes manually

ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])  # [left, bottom, width, height] 0-1
```

`add_axes` is used for **inset plots** (a small chart inside a big chart).

4 Line Plot + All Customization

▸ When to use

Line plot is for **trend over a continuous axis** — mostly time (date, year, month). Use it when order of points has meaning.

```
plt.plot(x, y)
plt.plot(df['date'], df['price'])
plt.plot(y)                     # if x is not given, x = 0,1,2,3...index
plt.plot(x, y1, x, y2)          # two lines in one call
```

▸ The famous format-string shortcut

```
plt.plot(x, y, 'ro--')   # r = red, o = circle marker, -- = dashed line
```

Colour	Marker	Line style
`b` blue, `g` green, `r` red, `c` cyan, `m` magenta, `y` yellow, `k` black, `w` white	`.` point, `o` circle, `s` square, `^` triangle, `*` star, `+` plus, `x` cross, `D` diamond	`-` solid, `--` dashed, `-.` dash-dot, `:` dotted

▸ Full keyword version (use this in real work)

```
plt.plot(x, y,
         color='#1f77b4',      # named / hex / (r,g,b) tuple
         linestyle='--',       # or ls
         linewidth=2,          # or lw
         marker='o',
         markersize=7,         # or ms
         markerfacecolor='white',
         markeredgecolor='red',
         alpha=0.8,            # transparency 0 to 1
         label='Sales 2024')   # needed for legend
```

▸ Decorating the plot (must-know 8 lines)

```
plt.title('Monthly Sales', fontsize=16, color='navy')
plt.xlabel('Month'); plt.ylabel('Sales in Lakhs')
plt.legend(loc='upper left')     # works only if label= was given
plt.xlim(0, 12); plt.ylim(0, 100)
plt.xticks(rotation=45)
plt.grid(True, linestyle=':', alpha=0.5)
plt.show()                       # renders + clears the current figure
```

WHY PLT.SHOW() AT THE END?

It flushes the figure to the backend and draws it. In a script, without `show()` nothing appears. In Jupyter with `%matplotlib inline` it is automatic, but writing it also hides the ugly `[<matplotlib.lines.Line2D ...>]` output line.

QUICK INTERVIEW POINT

Line plot of a **categorical** x-axis is wrong, because a line implies continuity between points. For categories, use a bar chart.

5 Scatter Plot

▸ When to use

To see the **relationship / correlation between two numerical columns**. Also to spot outliers and clusters.

```
plt.scatter(df['height'], df['weight'])
```

▸ Encoding more than 2 variables (very good interview answer)

A scatter plot can show **4 variables at once**:

Channel	Parameter	Shows
x position	`x`	numerical var 1
y position	`y`	numerical var 2
colour	`c` + `cmap`	var 3 (category or number)
size of dot	`s`	var 4 (number) — called a **bubble chart**

```
plt.scatter(df['height'], df['weight'],
            c=df['age'], cmap='viridis',   # colour = age
            s=df['salary']/1000,           # size = salary (bubble chart)
            alpha=0.7, marker='o',
            edgecolors='black', linewidths=0.5)
plt.colorbar(label='Age')                  # legend for the colour scale
```

PLT.PLOT VS PLT.SCATTER — ASKED A LOT

`plt.plot(x, y, 'o')` also gives dots, and it is **faster** for big data, but every dot must have the same colour and same size. `plt.scatter()` is slower but lets each dot have its own colour and size. So: same styling for all points → `plot`; different styling per point → `scatter`.

▸ Overplotting problem

With lakhs of points, dots sit on top of each other and the chart becomes a black blob. Fixes:

- `alpha=0.1` — transparency shows density
- `plt.hexbin(x, y, gridsize=30)` — hexagon density plot
- Take a random sample of rows
- Use a 2D histogram / KDE (Seaborn `kdeplot`)

6 Bar Chart (grouped, stacked, horizontal)

▸ When to use

To compare a **numerical value across categories** (sales per city, marks per student). Height of bar = value.

```
plt.bar(x=['A','B','C'], height=[10,25,17], color=['red','green','blue'], width=0.6)
plt.barh(['A','B','C'], [10,25,17])      # horizontal bar
```

WHEN TO PREFER BARH (HORIZONTAL)

When category names are long (city names, product names) or when there are many categories. Horizontal bars keep labels readable without rotation.

▸ Grouped bar chart (side by side)

Trick: shift the x positions manually using numpy.

```
import numpy as np
x = np.arange(len(months))      # 0,1,2,3...
w = 0.35
plt.bar(x - w/2, sales_2023, width=w, label='2023')
plt.bar(x + w/2, sales_2024, width=w, label='2024')
plt.xticks(x, months)           # put real names back on the ticks
plt.legend()
```

▸ Stacked bar chart

Trick: use the `bottom` parameter — the second bar starts on top of the first.

```
plt.bar(months, maths, label='Maths')
plt.bar(months, science, bottom=maths, label='Science')
plt.bar(months, english, bottom=np.array(maths)+np.array(science), label='English')
```

Type	Use when
Simple bar	Compare one value across categories
Grouped bar	Compare sub-groups **against each other** (2023 vs 2024)
Stacked bar	Show **parts of a total** and the total itself
100% stacked	Compare only the **proportion**, totals do not matter

▸ Adding value labels on bars

```
bars = plt.bar(x, y)
plt.bar_label(bars, padding=3, fmt='%.1f')   # modern one-liner
```

INTERVIEW TRAP

A bar chart's y-axis **must start from 0**. If it starts from 90, a small difference looks huge — that is a misleading chart. Line charts do not have this rule.

7 Histogram

▸ When to use

To see the **distribution of ONE numerical column** — where most values lie, is it skewed, are there outliers.

```
plt.hist(df['age'], bins=20, edgecolor='black')
plt.hist(df['age'], bins=[0,18,30,45,60,100])   # custom bin edges
plt.hist(df['age'], bins=20, density=True)      # y = probability density, area = 1
plt.hist(df['age'], cumulative=True)            # CDF shape
plt.hist([m_age, f_age], label=['Male','Female'])        # side by side
plt.hist(x, histtype='step')                    # outline only, good for overlap
```

▸ Histogram vs Bar chart — the most asked question

Point	Histogram	Bar chart
Data type on x	**Numerical** (continuous)	**Categorical**
x-axis meaning	Ranges / bins (0–10, 10–20)	Separate labels (Delhi, Mumbai)
Gap between bars	No gap (ranges are continuous)	Gap present
Bar order	Cannot be changed (number order)	Can be re-ordered freely
Y-axis	Frequency / count of values in that range	Any value of that category
Answers	"How is this column distributed?"	"Which category is bigger?"

▸ Bins — why they matter

- **Too few bins** → over-smoothed, you miss the real shape.
- **Too many bins** → noisy, spiky, hard to read.
- Default in Matplotlib is **10**. Common rules: Sturges, Freedman–Diaconis (`bins='auto'` uses these).

WHAT TO READ FROM A HISTOGRAM (SAY THIS)

- **Shape**: normal (bell), uniform, bimodal (2 peaks → two groups mixed).
- **Skew**: right/positive skew → long tail on right (income, price). Left skew → tail on left.
- **Spread** and **outliers**: lonely bars far away.

Related: KDE plot is the smooth version of a histogram (no bins). In Matplotlib you get KDE through Seaborn or SciPy.

8 Pie Chart

▸ When to use

To show **share / percentage of a whole**, when there are only 2–5 categories and they add up to 100%.

```
plt.pie(values, labels=names,
        autopct='%0.1f%%',          # show percentage on slices
        explode=[0, 0.1, 0, 0],     # pull out the 2nd slice
        shadow=True,
        startangle=90,
        colors=['#1f77b4','#ff7f0e','#2ca02c','#d62728'])
plt.pie(values, labels=names, wedgeprops={'width':0.4})   # donut chart
```

INTERVIEW ANSWER THAT IMPRESSES

"Pie charts are generally discouraged. Human eyes compare **lengths** better than **angles and areas**, so with more than 4–5 slices or slices of similar size, a pie chart becomes unreadable. I prefer a horizontal bar chart in those cases."

9 Box Plot & Other Statistical Plots

▸ Box plot (box-and-whisker)

Shows the **five-number summary** of a numerical column and its outliers, in very little space. Best for **comparing distributions across groups**.

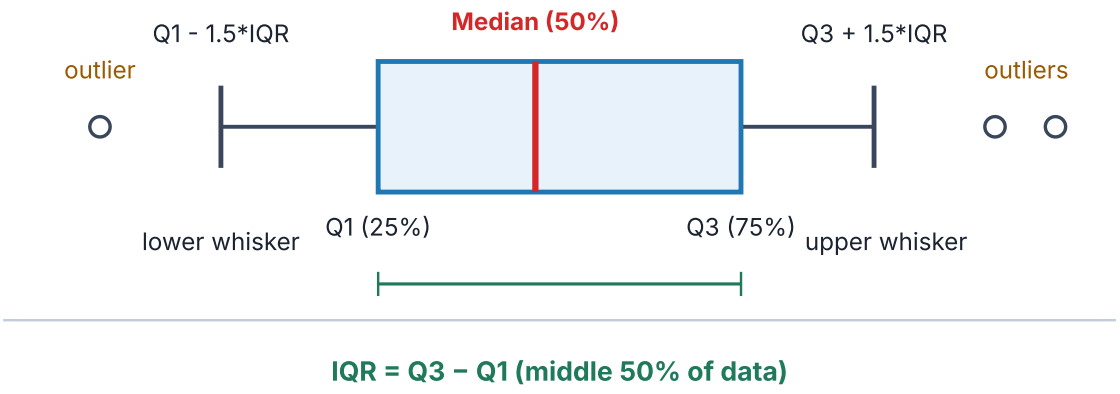


Figure 9.1 — Box plot anatomy. Anything beyond 1.5×IQR from the box is drawn as an outlier.

```
plt.boxplot(df['salary'])
plt.boxplot([s1, s2, s3], labels=['IT','HR','Sales'])
plt.boxplot(data, vert=False, showmeans=True, notch=True)
```

MUST-KNOW FORMULA

IQR = Q3 − Q1. Outlier if value < Q1 − 1.5×IQR or value > Q3 + 1.5×IQR. Whiskers go to the last real data point inside this range — **not** exactly to the limit.

▸ Box plot vs Histogram

Box plot	Histogram
Compact; easy to compare many groups side by side	Shows the real shape of the distribution
Marks outliers clearly	Outliers are just small bars
Cannot show bimodal (two peaks) — big weakness	Shows two peaks clearly

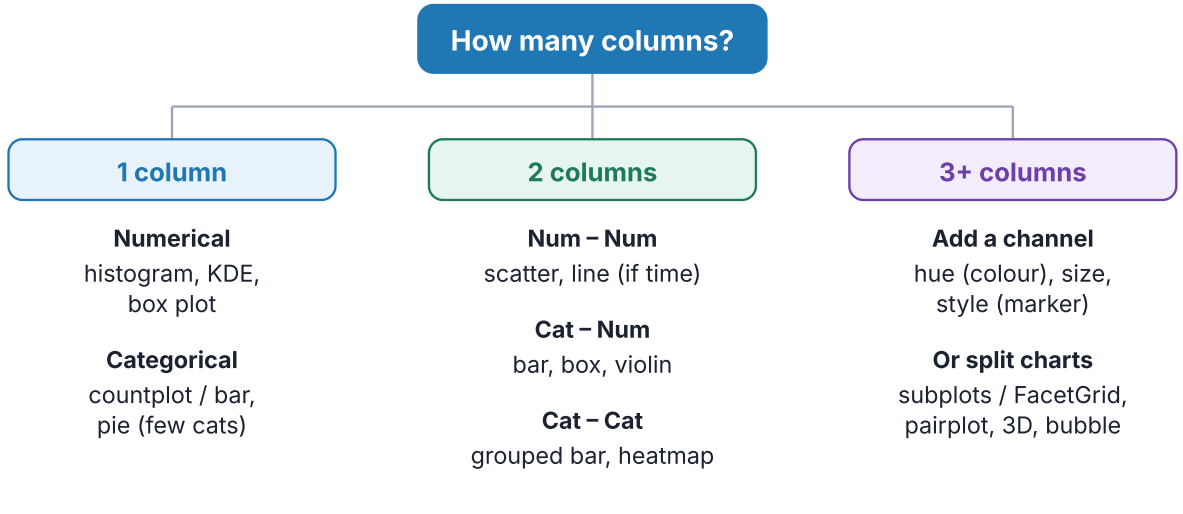
That weakness is why **violin plots** exist (box plot + KDE). Covered in the Seaborn doc.

▸ Other useful plots (short)

Function	What it does
`plt.stackplot(x, y1, y2)`	Area chart stacked — part-to-whole over time
`plt.fill_between(x, lo, hi)`	Shade a band (confidence interval, min–max range)
`plt.errorbar(x, y, yerr=e)`	Point + error bars (standard deviation / CI)
`plt.stem(x, y)`	Lollipop / signal plot — discrete values
`plt.hexbin(x, y, gridsize=25)`	2D density for huge scatter data
`plt.violinplot(data)`	Distribution shape + summary
`plt.step(x, y)`	Step chart — values that change suddenly
`plt.pcolormesh(X, Y, Z)`	2D colour grid for matrix data

10 Which Chart to Use When

This is a guaranteed interview question. Answer by first saying **how many variables** and **what type**, then name the chart.



Golden rule for the interview

First ask: how many variables, and are they numerical or categorical? The chart follows from that answer.

Figure 10.1 — Chart selection flow.

I want to show...	Best chart	Avoid
Trend over time	Line plot, area chart	Pie, bar with many points
Comparison of categories	Bar / horizontal bar	Pie if >5 categories
Distribution of 1 numerical	Histogram, KDE, box plot	Bar chart
Relation between 2 numerical	Scatter plot	Line (if not time-ordered)
Part of a whole	Stacked bar, pie/donut, treemap	Normal bar
Correlation of many columns	Heatmap of `df.corr()`	Many scatter plots
Composition + total over time	Stacked area chart	Many lines
Outlier detection	Box plot, scatter	Pie
Spread by group	Box plot / violin by category	Only mean in a bar chart

11 Subplots & Layout

Subplot = many Axes inside one Figure. There are 4 ways, and interviewers ask the difference.

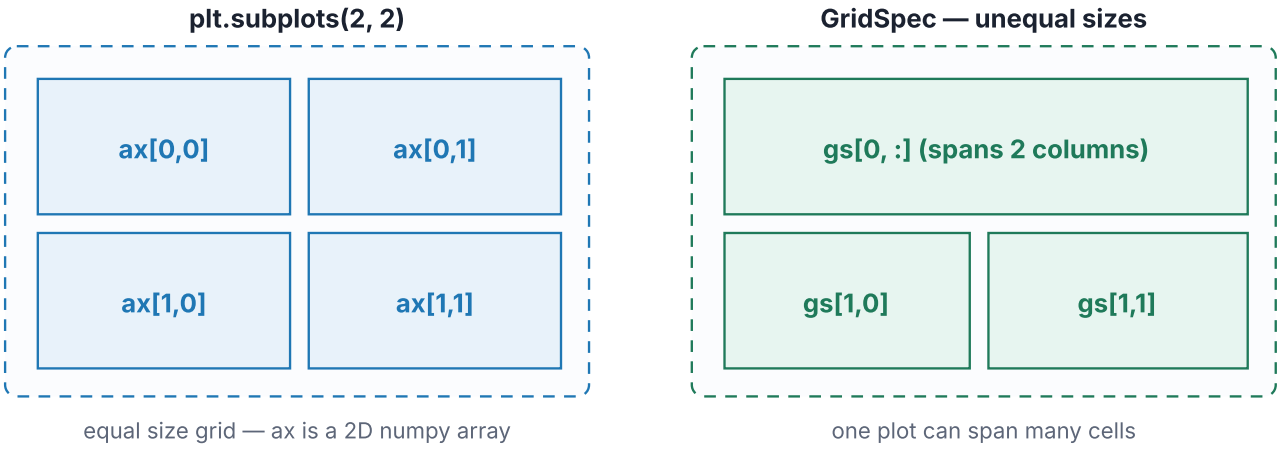


Figure 11.1 — Regular grid vs GridSpec.

▸ Way 1 — plt.subplots() (use this 90% of time)

```
fig, ax = plt.subplots(nrows=2, ncols=2, figsize=(12,8),
                       sharex=True, sharey=True)
ax[0,0].plot(x, y);        ax[0,0].set_title('Line')
ax[0,1].scatter(x, y);     ax[0,1].set_title('Scatter')
ax[1,0].hist(data)
ax[1,1].bar(cat, val)
fig.suptitle('My Dashboard', fontsize=18)   # title of the whole FIGURE
plt.tight_layout()                          # fix overlapping labels
```

If `nrows=1` or `ncols=1`, `ax` is a **1D** array → use `ax[0]`, not `ax[0,0]`. With 1×1 it is a single object, not an array at all.

▸ Way 2 — plt.subplot() (old pyplot style, 1-indexed)

```
plt.subplot(2, 2, 1)   # rows, cols, index (starts at 1, left to right)
plt.plot(x, y)
plt.subplot(2, 2, 2)
plt.hist(data)
```

▸ Way 3 — fig.add_subplot()

```
fig = plt.figure(figsize=(10,6))
ax1 = fig.add_subplot(2, 2, 1)
ax2 = fig.add_subplot(2, 2, 2)
```

Useful when subplots must be added one by one, or for 3D: `fig.add_subplot(projection='3d')`.

▸ Way 4 — GridSpec / subplot2grid (unequal sizes)

```
import matplotlib.gridspec as gridspec
fig = plt.figure(figsize=(10,6))
gs  = gridspec.GridSpec(2, 2, height_ratios=[2,1], hspace=0.3, wspace=0.2)
ax1 = fig.add_subplot(gs[0, :])    # full top row
ax2 = fig.add_subplot(gs[1, 0])
ax3 = fig.add_subplot(gs[1, 1])

# older equivalent
ax1 = plt.subplot2grid((3,3), (0,0), colspan=3)
ax2 = plt.subplot2grid((3,3), (1,0), rowspan=2, colspan=2)
```

Parameter	Meaning
`figsize=(w,h)`	Figure size in **inches**
`dpi=100`	Dots per inch. Pixels = figsize × dpi
`sharex / sharey`	Same axis range for all subplots; inner tick labels are hidden
`tight_layout()`	Auto-adjust spacing so nothing overlaps
`constrained_layout=True`	Newer, smarter version of tight_layout
`hspace / wspace`	Vertical / horizontal gap between subplots
`fig.suptitle()`	Title for the whole figure (vs `ax.set_title()` for one subplot)

COMMON INTERVIEW TASK

"Plot 4 charts in one figure." Answer: `fig, ax = plt.subplots(2, 2, figsize=(12,8))`, then draw on `ax[0,0]` … `ax[1,1]`, and finish with `plt.tight_layout()`. Mention `sharey=True` if the charts must be compared on the same scale.

12 Colors, Colormaps, Styles, rcParams

▸ Ways to give a colour

```
color='red'              # 1. name
color='r'                # 2. single letter
color='#1f77b4'          # 3. hex
color=(0.1, 0.4, 0.8)    # 4. RGB tuple, 0-1
color=(0.1,0.4,0.8,0.5)  # 5. RGBA (last = alpha)
color='C0'               # 6. C0..C9 = default colour cycle
color='tab:blue'         # 7. tableau palette
```

▸ Colormaps (cmap) — 3 types

Type	Use when	Examples
Sequential	Values go low → high (one direction)	`viridis`, `plasma`, `Blues`, `YlOrRd`
Diverging	There is a meaningful middle (0, average). E.g. correlation −1 to +1	`coolwarm`, `RdBu`, `bwr`, `seismic`
Qualitative	Categories with no order	`tab10`, `Set1`, `Paired`

SMART POINT TO MENTION

"I prefer `viridis` over `jet`. `jet` is not perceptually uniform — equal steps in data do not look like equal steps in colour, and it is bad for colour-blind users. `viridis` is uniform and colour-blind friendly, that is why it is the Matplotlib default."

▸ Styles — change the whole look in one line

```
print(plt.style.available)
plt.style.use('ggplot')        # seaborn-v0_8, fivethirtyeight, dark_background, bmh, classic
with plt.style.context('dark_background'):   # apply to one plot only
    plt.plot(x, y)
```

▸ rcParams — global default settings

```
plt.rcParams['figure.figsize']  = (10, 5)
plt.rcParams['font.size']       = 12
plt.rcParams['axes.grid']       = True
plt.rcParams['lines.linewidth'] = 2
plt.rcParams.update({'axes.titlesize': 16})
plt.rcdefaults()               # reset everything back
```

rc = "runtime configuration". These defaults come from a `matplotlibrc` file. Good answer when asked "how do you avoid writing figsize again and again?"

13 Text, Annotation & Legend Control

▸ Text and arrows

```
plt.text(x=5, y=20, s='Peak sales', fontsize=12, color='red',
         ha='center', va='bottom',
         bbox=dict(facecolor='yellow', alpha=0.4, boxstyle='round'))

plt.annotate('Highest point',
             xy=(5, 25),                # point to mark
             xytext=(7, 30),            # where the text sits
             arrowprops=dict(arrowstyle='->', color='black', lw=1.5))

plt.axhline(y=50, color='red', ls='--', label='Target')   # horizontal line
plt.axvline(x=10, color='grey')                            # vertical line
plt.axhspan(40, 60, color='green', alpha=0.1)              # shaded band
```

`text()` only writes text. `annotate()` writes text **and draws an arrow** to a data point — that is the difference they ask.

▸ Legend control

```
plt.plot(x, y, label='2024')
plt.legend(loc='best',            # upper right/left, lower.., center, best
           fontsize=10,
           title='Year',
           ncol=2,                # legend in 2 columns
           frameon=False,
           bbox_to_anchor=(1.02, 1))   # push legend OUTSIDE the plot

# manual legend when labels are not set
ax.legend(['first line', 'second line'])
```

WHY IS MY LEGEND EMPTY / NOT SHOWING?

Because `label=` was not passed to the plotting function. `plt.legend()` only collects artists that have a label. (A label starting with `_` is also skipped.)

14 Axis Control: limits, ticks, spines, twin, log

▸ Limits and ticks

```
plt.xlim(0, 100); plt.ylim(0, 50)
ax.set_xlim(0, 100)                        # OO style
plt.xticks([0,25,50,75,100], ['0','25','50','75','100'], rotation=45)
ax.set_xticks(ticks); ax.set_xticklabels(labels)
ax.tick_params(axis='x', labelsize=9, colors='grey', length=6)
ax.invert_yaxis()                           # flip the axis
```

▸ Spines — remove the ugly box

```
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_position(('outward', 10))
```

In Seaborn this one line does the same: `sns.despine()`.

▸ Log scale

```
plt.yscale('log')      # also 'linear', 'symlog', 'logit'
ax.set_xscale('log')
```

Use log scale when values span many orders of magnitude (1, 100, 10000) or data has exponential growth — on a log axis exponential growth becomes a straight line.

▸ Twin axis — two different units in one chart

```
fig, ax1 = plt.subplots()
ax1.plot(months, sales, color='blue')
ax1.set_ylabel('Sales (Crore)', color='blue')

ax2 = ax1.twinx()                # shares the x-axis, new y-axis on right
ax2.plot(months, temperature, color='red')
ax2.set_ylabel('Temperature (C)', color='red')
```

`twinx()` = share x, separate y. `twiny()` = share y, separate x. Mention the caution: twin axes can mislead, because the scales are chosen by hand.

▸ Date axis

```
import matplotlib.dates as mdates
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b-%Y'))
ax.xaxis.set_major_locator(mdates.MonthLocator())
fig.autofmt_xdate()             # auto-rotate date labels
```

15 Images & Heatmap with imshow

An image is just a NumPy array: grayscale is 2D `(h, w)`, colour is 3D `(h, w, 3)` for RGB or `(h, w, 4)` for RGBA.

```
img = plt.imread('photo.png')     # numpy array
print(img.shape)                  # (height, width, channels)
plt.imshow(img)
plt.axis('off')

# grayscale / matrix as heatmap
plt.imshow(matrix, cmap='viridis', interpolation='nearest', aspect='auto')
plt.colorbar(label='value')

# correlation heatmap in pure matplotlib
plt.imshow(df.corr(), cmap='coolwarm', vmin=-1, vmax=1)
plt.xticks(range(len(df.columns)), df.columns, rotation=90)
plt.yticks(range(len(df.columns)), df.columns)
```

Parameter	Meaning
`cmap`	Colormap used to map numbers to colours
`vmin, vmax`	Fix the colour scale range (important for fair comparison)
`interpolation`	Smoothing between pixels (`'nearest'` = no smoothing)
`aspect`	`'equal'` keeps square pixels, `'auto'` stretches to the axes
`origin`	`'upper'` (default, row 0 on top) or `'lower'`

In practice everyone uses `sns.heatmap()` because it adds annotations and a colorbar automatically.

16 3D Plotting

```
fig = plt.figure(figsize=(9,6))
ax  = fig.add_subplot(projection='3d')      # this one line makes it 3D

ax.scatter3D(x, y, z, c=z, cmap='viridis')
ax.plot3D(x, y, z)
ax.set_xlabel('X'); ax.set_ylabel('Y'); ax.set_zlabel('Z')
ax.view_init(elev=30, azim=45)              # camera angle
```

▸ Surface / wireframe / contour — needs meshgrid

```
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)        # 1D arrays -> 2D coordinate grids
Z = np.sin(np.sqrt(X**2 + Y**2))

ax.plot_surface(X, Y, Z, cmap='viridis')
ax.plot_wireframe(X, Y, Z, color='grey')
ax.contour3D(X, Y, Z, 50)
plt.contourf(X, Y, Z, cmap='coolwarm')      # 2D filled contour
```

WHY MESHGRID? (ASKED WITH 3D AND WITH ML DECISION BOUNDARIES)

"Z must be computed at every (x, y) pair. `meshgrid` turns two 1D arrays into two 2D grids so that `X[i][j], Y[i][j]` gives the coordinate of every cell. Then Z = f(X, Y) can be computed in one vectorized step. The same trick is used to draw ML decision boundaries."

Interview caution: 3D charts look cool but hide data behind data. In real reports, 2D with colour/size encoding is usually better.

17 Saving Figures, DPI & Backends

```
plt.savefig('chart.png',
            dpi=300,                # 300 for print, 100 for web
            bbox_inches='tight',    # remove extra white border
            transparent=True,
            facecolor='white')
fig.savefig('chart.pdf')            # OO style (preferred)
```

CLASSIC BUG: SAVED IMAGE IS BLANK

Because `plt.show()` was called **before** `plt.savefig()`. `show()` clears the current figure. Always **save first, then show**.

▸ PNG vs SVG vs PDF

Format	Type	Use for
PNG / JPG	Raster (pixels)	Web, PPT, notebooks. Blurs on zoom.
SVG / PDF / EPS	Vector	Reports, research papers. Sharp at any zoom.

▸ Backends & Jupyter magics

Command	Meaning
`%matplotlib inline`	Static image inside the notebook (default)
`%matplotlib notebook` / `widget`	Interactive chart (zoom, pan) inside the notebook
`matplotlib.use('Agg')`	Non-GUI backend. Needed on servers with no display — for saving files only.
`plt.close()` / `plt.close('all')`	Free memory in loops that create many figures

"**Agg**" = Anti-Grain Geometry. Interview-worthy: if a plotting script crashes on a Linux server with "no display name", the fix is `matplotlib.use('Agg')`.

18 Animation (FuncAnimation)

```
from matplotlib.animation import FuncAnimation

fig, ax = plt.subplots()
line, = ax.plot([], [])             # empty line; comma unpacks the list
ax.set_xlim(0, 10); ax.set_ylim(-1, 1)

def update(frame):
    x = np.linspace(0, 10, 200)
    y = np.sin(x + frame/10)
    line.set_data(x, y)             # update data, do not re-plot
    return line,

ani = FuncAnimation(fig, update, frames=100, interval=50, blit=True)
ani.save('wave.gif', writer='pillow', fps=20)
```

Parameter	Meaning
`frames`	How many frames (or a list of values passed to `update`)
`interval`	Delay between frames in milliseconds (50 ms = 20 FPS)
`blit=True`	Redraw only what changed → much faster
`repeat`	Loop the animation or stop at the end

Save as GIF with `pillow`, as MP4 with `ffmpeg`. `ArtistAnimation` is the other class: you pre-build a list of frames instead of a function.

19 Matplotlib vs Seaborn vs Plotly vs Pandas .plot()

Library	Built on	Strength	Weakness
Matplotlib	—	Full control over every element; works everywhere; can save any format	Verbose; default look is plain; no statistics
Seaborn	Matplotlib	Beautiful defaults; statistical plots in 1 line; works directly with DataFrames (`hue`, `col`)	Less low-level control; for fine tuning you fall back to Matplotlib
Pandas .plot()	Matplotlib	Fastest for quick EDA: `df.plot(kind='bar')`	Very limited customization
Plotly	plotly.js (web)	Interactive: hover, zoom, dropdown; good for dashboards (Dash/Streamlit)	Heavier; big HTML output

BEST ANSWER FOR "MATPLOTLIB OR SEABORN?"

"They are not competitors — Seaborn is a wrapper built on top of Matplotlib. I use Seaborn to get a good statistical chart quickly in one line, and then use Matplotlib functions on the same axes to fine-tune titles, limits and annotations. Every Seaborn axes-level function returns a Matplotlib Axes object, so both mix freely."

20 Common Errors + Cheat Sheet

Problem	Reason & fix
Chart does not appear in a .py script	`plt.show()` missing
Saved file is blank	`savefig()` called after `show()`. Save first.
Labels are cut off in the saved image	Use `bbox_inches='tight'` or `plt.tight_layout()`
Legend not showing	`label=` not given in the plot call
`TypeError: 'AxesSubplot' object is not subscriptable`	Only one subplot exists, so `ax` is not an array. Use `ax`, not `ax[0]`.
Charts keep stacking on each other	Same current figure is being reused. Call `plt.figure()` for a new one or use the OO interface.
Memory warning after many figures	Add `plt.close()` inside the loop
x-axis labels overlapping	`plt.xticks(rotation=45)` or use `barh`
"No display name" error on a server	`matplotlib.use('Agg')` before importing pyplot

▸ Best practices (good closing lines in an interview)

- Always label axes and add units; a chart without labels is useless.
- Use the OO interface for anything bigger than a quick look.
- Bar chart y-axis must start at 0.
- Use colour-blind-friendly colormaps (`viridis`); do not use colour as the only signal.
- Remove chart junk: extra gridlines, 3D effects, heavy borders.
- One chart should answer one question.

▸ Matplotlib Cheat Sheet (last-minute revision)

Setup

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(10,5))
```

Plot types

```
ax.plot(x,y)          # line
ax.scatter(x,y)       # scatter
ax.bar(x,h)/barh      # bar
ax.hist(v,bins=20)    # histogram
ax.pie(v,labels=l)    # pie
ax.boxplot(d)         # box
ax.violinplot(d)      # violin
ax.stackplot(x,y1,y2) # area
ax.fill_between(x,a,b)# band
ax.errorbar(x,y,yerr=e)
ax.imshow(mat)        # heatmap
ax.hexbin(x,y)        # density
```

Labels

```
ax.set_title('t')
ax.set_xlabel('x'); ax.set_ylabel('y')
ax.legend(loc='best')
ax.grid(alpha=.4)
fig.suptitle('all')
```

Axis

```
ax.set_xlim(0,10)
ax.set_xticks(t); ax.set_xticklabels(l)
plt.xticks(rotation=45)
ax.set_yscale('log')
ax2 = ax.twinx()
ax.spines['top'].set_visible(False)
```

Style

```
color/c, linestyle/ls, linewidth/lw,
marker, markersize/ms, alpha, label
plt.style.use('ggplot')
plt.rcParams['figure.figsize']=(10,5)
cmap='viridis' (seq) 'coolwarm' (div)
            'tab10' (categorical)
```

Subplots & save

```
fig, ax = plt.subplots(2,2, sharey=True)
ax[0,0].plot(...)
gridspec.GridSpec(2,2,height_ratios=[2,1])
plt.tight_layout()
plt.savefig('a.png', dpi=300,
            bbox_inches='tight')   # BEFORE
show
plt.show(); plt.close()
```

Annotate

```
ax.text(x,y,'s')
ax.annotate('s', xy=(x,y), xytext=(a,b),
      arrowprops=dict(arrowstyle='->'))
ax.axhline(50, ls='--')
```

21 Real Interview Questions & Answers

Answers are written the way you can actually speak them — short, simple, no heavy English.

▸ A. Basic / theory

1. What is Matplotlib and why is it used?

It is Python's base plotting library. It is used to make charts for EDA and reporting — line, bar, scatter, histogram and more. Other libraries like Seaborn and Pandas plotting are built on top of it, so it is the foundation of visualization in Python.

2. Explain the architecture of Matplotlib.

Three layers. Backend layer actually draws the output (Agg for PNG, PDF for files, TkAgg for windows). Artist layer holds all objects we see — Figure, Axes, Line2D, Text; each one is an Artist. Scripting layer is `pyplot`, the simple interface we normally write.

3. Difference between Figure, Axes and Axis?

Figure is the whole canvas, like a page. Axes is one plot drawn on that page — it has the data, title and labels. Axis is the x or y number line of that plot, which controls ticks and limits. One Figure can hold many Axes, and one Axes has two Axis objects.

4. What is the difference between pyplot interface and object-oriented interface?

pyplot is a state machine — it remembers the "current figure" and all commands go there automatically. In the OO interface I get the Figure and Axes objects in variables and call methods on them, like `ax.set_title()`. For a quick chart pyplot is fine, but for multiple subplots or a reusable function the OO way is safer and clearer.

5. What does plt.show() do, and why is my plot not visible?

It sends the figure to the backend and displays it, and then clears the current figure. In a Python script, nothing appears without it. In Jupyter with `%matplotlib inline` plots show automatically.

6. What is an Artist in Matplotlib?

Anything visible on the figure is an Artist object — the Figure itself, the Axes, a line, a tick, a text, a legend. There are two kinds: containers (Figure, Axes) and primitives (Line2D, Text, Patch).

7. What is a backend? Which one do you use on a server?

The backend is the engine that actually renders the chart. On a server with no screen I use `matplotlib.use('Agg')` before importing pyplot — it is a non-GUI backend that can only write files, which is exactly what is needed there.

▸ B. Charts and chart selection

8. Difference between a histogram and a bar chart?

Histogram is for one numerical column — the x-axis has ranges (bins) and the y-axis is the count of values in that range, so bars touch each other. A bar chart is for categorical data — each bar is a separate category with a gap, and the bars can be reordered. Histogram answers "how is the data distributed", bar chart answers "which category is bigger".

9. What is the effect of changing the number of bins?

Too few bins hide the real shape of the data; too many bins make it noisy and spiky. Default is 10, and `bins='auto'` picks a good number using statistical rules like Freedman–Diaconis.

10. When would you use a line plot instead of a bar chart?

When the x-axis is continuous and ordered, mostly time. A line shows trend and connects points, which only makes sense if the points are in order. For separate categories I use bars.

11. plt.plot(x, y, 'o') vs plt.scatter(x, y) — which is better?

`plot` with a marker is faster for large data, but all points share the same colour and size. `scatter` is slower but allows a different colour and size for every point, so I use it when I want to encode extra variables through `c` and `s`.

12. How do you show 4 variables in one 2D chart?

With a scatter plot: x position, y position, colour (`c` with a cmap) and dot size (`s`). Size encoding makes it a bubble chart. If a variable is categorical, I can use marker shape or make subplots per category.

13. Why are pie charts not recommended?

Because people judge lengths better than angles and areas. With more than four or five slices, or slices of similar size, it becomes hard to read. A horizontal bar chart shows the same information more clearly.

14. How do you read a box plot? What is IQR?

The box shows Q1, median and Q3, so the box itself is the middle 50% of the data. IQR = Q3 − Q1. Whiskers extend up to 1.5×IQR, and points beyond that are drawn as outliers. It is best for comparing many groups side by side, but it cannot show two peaks — for that I use a histogram or a violin plot.

15. Too many points are overlapping in my scatter plot. What do you do?

That is overplotting. I reduce `alpha` so density becomes visible, or use `hexbin` or a 2D KDE, or plot a random sample of the rows.

▸ C. Code / practical

16. How do you plot 4 charts in one figure?

`fig, ax = plt.subplots(2, 2, figsize=(12,8))`, then draw on `ax[0,0]` to `ax[1,1]`, and finally `plt.tight_layout()`. If the charts must be compared on the same scale, I add `sharey=True`.

17. Difference between plt.subplot() and plt.subplots()?

`plt.subplots()` (with s) creates the figure and all the axes at once and returns `fig, ax`. `plt.subplot()` creates or selects one axes at a time using 1-based indexing, like `plt.subplot(2,2,1)`. The first one is the modern way.

18. How do you make subplots of different sizes?

With GridSpec. I create `gs = gridspec.GridSpec(2, 2, height_ratios=[2,1])` and then use slicing like `fig.add_subplot(gs[0, :])` so one plot spans the whole top row. `plt.subplot2grid` with `rowspan` / `colspan` does the same thing.

19. Difference between fig.suptitle() and ax.set_title()?

`suptitle` is the main title of the whole figure; `set_title` is the title of one subplot.

20. What does tight_layout() do?

It automatically adjusts the spacing and padding so titles and tick labels do not overlap or get cut off. `constrained_layout=True` is the newer and smarter version.

21. How do you plot two different units on one chart?

With a twin axis: `ax2 = ax1.twinx()` shares the x-axis and adds a second y-axis on the right. I colour each y-label to match its line so the reader knows which axis belongs to which line.

22. How do you save a figure properly?

`plt.savefig('x.png', dpi=300, bbox_inches='tight')`, and always **before** `plt.show()`, because `show()` clears the figure and you get a blank image. For reports I save PDF or SVG because they are vector formats and stay sharp on zoom.

23. figsize and dpi — what is the relation?

`figsize` is in inches and `dpi` is dots per inch, so final pixels = figsize × dpi. For example `figsize=(8,4)` with `dpi=100` gives an 800×400 image.

24. How do you change default settings for all plots?

Through `plt.rcParams`, for example `plt.rcParams['figure.figsize'] = (10,5)`. For the whole look at once I use `plt.style.use('ggplot')`. `plt.rcdefaults()` resets everything.

25. Difference between text() and annotate()?

`text()` just writes text at a position. `annotate()` writes text and can draw an arrow from the text to a specific data point using `xy`, `xytext` and `arrowprops`.

26. My legend is not appearing. Why?

Because I did not pass `label=` in the plotting call. `plt.legend()` only picks up artists that have a label.

27. What are the types of colormaps and when do you use each?

Sequential for low-to-high values like `viridis` or `Blues`. Diverging when there is a meaningful centre, like a correlation matrix from −1 to +1, using `coolwarm` or `RdBu`. Qualitative for unordered categories, like `tab10`. I avoid `jet` because it is not perceptually uniform and is bad for colour-blind users.

28. What is meshgrid and why is it needed for 3D plots?

`np.meshgrid` converts two 1D arrays into two 2D coordinate grids, so every cell has its own (x, y) pair. Then Z = f(X, Y) can be computed for the whole surface in one vectorized operation. `plot_surface` and `contour` need this 2D form.

29. Matplotlib vs Seaborn — which one do you use?

Both together. Seaborn is built on Matplotlib and gives beautiful statistical plots in one line with DataFrame support like `hue` and `col`. But for fine control — limits, annotations, custom ticks — I use Matplotlib on the same axes object that Seaborn returns.

30. How do you create an animated chart?

With `FuncAnimation`. I create the figure and an empty line, write an `update(frame)` function that calls `line.set_data()` with new values, and pass it to `FuncAnimation(fig, update, frames=100, interval=50, blit=True)`. Then `ani.save('x.gif', writer='pillow')` saves it.

31. Your chart will be shown to a business team. What would you take care of?

Clear title that states the insight, labelled axes with units, no chart junk, bar charts starting at 0, limited colours with a colour-blind-safe palette, and the most important number annotated directly on the chart. One chart should answer one question.

LAST TIP

In a visualization interview, never answer only with the function name. Say **why** that chart — "two numerical columns, so scatter", "one numerical column, so histogram". That single habit makes the answer sound experienced.